

Quiz 5: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 5 is Thursday Oct 1, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

Scope: PyTorch and training

Everything in the Week 5 deck, plus Lab 4. This is the first quiz with code on it, and the first that assumes you have written a training loop yourself.

On the quiz:

- Shapes, and broadcasting, and the shape bug that does not raise
- `.grad`, why it accumulates, and what `zero_grad()` is for
- `model.train()` against `model.eval()`, and how both differ from `torch.no_grad()`
- Logits, `CrossEntropyLoss`, and what does **not** go on the end of the model
- The five lines of the training loop, and what each one does
- Batch size: steps per epoch, and how gradient noise scales
- `Dataset` and `DataLoader`, and what `num_workers` is actually for
- Train, validation and test, and what leakage is
- Optimizers: SGD, momentum, RMSProp, Adam, and the learning rates they want
- Learning-rate schedules, warmup, and gradient clipping
- Xavier and He initialization, and why He uses the larger variance
- Early stopping, weight decay, dropout, and **Adam** against **AdamW**

On the quiz from Lab 4, which was due the day before:

- When `num_workers` helps and when it makes things **slower**
- Why the validation split is carved out of train and never out of test
- Why you overfit 100 examples before starting a real run
- Why early stopping needs a **cloned** copy of the weights

Format. This one is different from Quizzes 2 through 4, because the material is different. Part A is true or false and Part B is multiple choice, as usual. Part C replaces multiple answer with **fill in the blank**: you compute a number and write it down. Part D replaces short answer with **find the bugs**: you are given a training script that runs without error and trains badly, and you find three things wrong with it.

Nothing on the quiz asks you to write code from scratch. Part D asks you to read it.

Part 1: Tensors, shapes, and autograd

The shape bug that does not raise

Broadcasting aligns shapes **from the right**, treats a missing axis as size 1, and stretches any axis of size 1 to match. That is usually what you want. Once in a while it is a disaster:

$$(N, 1) - (N,) \rightarrow (N, N)$$

A model that outputs $(32, 1)$ and labels that are $(32,)$ produce a 32×32 matrix of **every pairwise difference**, not the 32 differences you meant. Raw tensor subtraction gives no warning at all. The cost that comes out is a real number of a believable size, and the model simply never learns. In the lecture example the cost was 9.7 times too large.

`nn.MSELoss` does emit a `UserWarning` for this case. A loss you write yourself gives you nothing. Do not rely on the warning.

The fix is `y.unsqueeze(1)` or `pred.squeeze(1)`. The habit is to **print shapes, not values**.

Be able to work out the result shape of a broadcast for yourself; the rule is on the formula sheet, the application is the question.

`.grad` accumulates

`.grad` is a **running sum**. Call `.backward()` twice without clearing and the second gradient is added to the first:

after	<code>.grad</code>
first <code>backward()</code>	-2.08
second <code>backward()</code>	-4.15
third <code>backward()</code>	-6.23

This is not a design flaw. It is the same rule that makes a fork in the computational graph work, from Quiz 4: a node reached along two paths adds both contributions. It is also useful on purpose, when you want to simulate a large batch on a small GPU by accumulating over several small ones.

The consequence for you is `optimizer.zero_grad()` at the top of every batch. Leave it out and each step uses the sum of every gradient so far in the epoch, the effective step size grows batch after batch, and the run goes unstable. Nothing raises.

`eval()` and `no_grad()` are different things

This is a favourite exam question because the two are almost always used together and do completely unrelated jobs.

Call	What it does	What breaks without it
<code>model.eval()</code>	Switches dropout off and makes batch norm use its running statistics	Validation runs with dropout still firing, so the number is worse than the model really is, and different every time
<code>torch.no_grad()</code>	Stops autograd recording the graph	Nothing is wrong numerically; you waste memory and time storing activations for a backward pass that never comes

Neither one does the other's job. `model.eval()` does **not** stop gradients from being computed.

Part 2: Models, losses, and the loop

Logits, and what does not go on the end

`nn.CrossEntropyLoss` applies `logsoftmax` internally and then takes the negative log-likelihood. So:

- The model's last layer emits **logits**, raw scores, shape (N, C)
- The targets are **integer class labels**, shape (N,), dtype `int64`, not one-hot
- You do **not** put a softmax on the end

Put one there anyway and it gets applied twice. The gradients shrink, training crawls, and nothing raises: on a random batch the correct loss was 2.71 and the double-softmaxed one was 2.27. Both look like losses.

The reason the softmax is folded in is that the combination collapses. For the softmax followed by cross-entropy, the gradient at the logits is just

$$\frac{\partial \mathcal{L}}{\partial z_i} = p_i - y_i$$

the predicted probability minus the target. Same shape of result as the sigmoid and binary cross-entropy pairing from Quiz 4, and for the same reason.

The five lines

```
optimizer.zero_grad()      # 1. clear the running sum
loss = criterion(model(xb), yb) # 2 and 3. forward, then score it
loss.backward()            # 4. fill every .grad
optimizer.step()           # 5. move every parameter
```

Know what each line does and what happens if it is missing or out of order. Line 4 is the machinery you wrote by hand in Lab 3.

Batching arithmetic

$$\text{iterations per epoch} = \left\lceil \frac{N}{B} \right\rceil$$

One **optimizer step** is one parameter update. Practise turning a dataset size and a batch size into a step count, including over several epochs, because the quiz asks you to.

The trap worth naming: **evaluation takes zero optimizer steps**. A validation pass runs forward passes and computes a loss, and updates nothing. If a question asks for optimizer steps during validation, the answer is 0 regardless of how the validation set is batched.

$N = 100$	Batch size	Steps per epoch	Gradient noise
Full batch	100	1	none
Mini-batch	25	4	moderate
Stochastic	1	100	very high

How gradient noise scales

The standard error of the gradient estimate goes like $1/\sqrt{B}$. Read the consequences off that, because they are not the intuitive ones:

- Doubling the batch **doubles** the work per step and reduces the noise by a factor of $\sqrt{2}$, about **29%**. Not half
 - To **halve** the noise you need **four times** the batch
 - So large batches have sharply diminishing returns, and small batches are not merely tolerable. The noise helps the optimizer rattle out of sharp minima and saddle points
-

Part 3: Data

Dataset and DataLoader

A **Dataset** is a **recipe for producing one training example**, not a container. Three methods: `__init__` says where the data lives, `__len__` says how many there are, and `__getitem__(i)` **produces example i**, doing the type conversion, the scaling, the reshape, and on real image data the decode and the augmentation. It runs once per example per epoch.

A **DataLoader** batches, shuffles, collates, and **loads in the background**. The fourth one is why it exists. If the model takes 20 ms per batch and the loader takes 40 ms to build one, your hardware sits idle two thirds of the time and no amount of model tuning fixes it.

When `num_workers` helps, and when it hurts

Lab 4 measured both directions, and this is the part worth remembering:

Dataset	<code>num_workers=0</code>	<code>num_workers=2</code>
In-memory tensors, <code>__getitem__</code> is a slice	113,000 img/s	75,000 img/s
<code>__getitem__</code> does real work, like a JPEG decode	4,500 img/s	13,400 img/s (with 4 workers)

Workers **hurt** the first one. There is nothing to parallelize, and you have paid for process startup and for shipping every batch back through a pipe. They roughly **triple** the second, because there is real work to overlap.

Workers help exactly when `__getitem__` is slow. The rule is not “more is better,” and the answer is not knowable without measuring.

The diagnostic to recognize: GPU utilization that keeps collapsing to 0% and spiking back means the GPU is waiting on data. Raise `num_workers`. Nothing you change about the model will help, because the model is not the bottleneck.

Two more settings: `pin_memory=True` makes the host-to-GPU copy faster and is useless on CPU, and `persistent_workers=True` keeps workers alive between epochs.

Train, validation, test, and leakage

- **Train** is what the optimizer sees
- **Validation** is what you see every epoch, to choose when to stop and which settings to keep
- **Test** is what nobody sees until the end, and you look once

The validation set is carved out of **train**, never out of test.

Leakage is when information that should only be in the training set finds its way into training anyway. The validation score then measures memorization rather than generalization, so it looks excellent and the model is useless on new data. Four common ways:

1. **Normalizing before splitting**, so the mean and standard deviation you subtracted contain the validation set
2. **Splitting randomly when the data is grouped**, putting the same patient or user on both sides
3. **Splitting randomly when the data is a time series**, training on the future to predict the past
4. **Duplicates**, near-identical rows on both sides

How to spot it: validation cost suspiciously close to training cost, or a jump in performance right after you added a preprocessing step.

Part 4: Optimizers, schedules, and clipping

Why one learning rate is not enough

A single η has to work for every parameter, and different parameters need very different step sizes. Some need small steps or the cost blows up; others barely move. The adaptive methods exist to fix that.

Optimizer	What it adds	Typical rate
SGD	nothing; the baseline	10^{-1}
SGD(momentum=0.9)	a velocity, so consistent directions build speed and oscillating ones cancel	10^{-1}
RMSprop	per-parameter scaling by recent gradient size	10^{-3}
Adam	momentum and per-parameter scaling	10^{-3}
AdamW	Adam, with weight decay implemented correctly	10^{-3}

Know that the rates differ by two orders of magnitude, and why: Adam has already divided the gradient by its running size before η is applied. A rate of 0.1 copied out of an SGD script into Adam will **diverge**. Nothing raises. This is one of the most common ways a working script stops working.

What Lab 4 found, which is worth carrying: the optimizer mattered a great deal for **how fast** the training loss fell, and very little for the **best validation number** any of them reached. Faster optimization is not better generalization.

Schedules, warmup, and clipping

Dropping the learning rate partway through is usually worth more than switching optimizers. On Lab 4's setup, cosine annealing moved validation accuracy from 0.844 to 0.863 with no other change.

Gradient clipping puts a ceiling on how large one update can be, so a single bad batch cannot destroy a model that took hours to train. The distinction to know:

- `clip_grad_norm_` rescales the **whole vector** by one scalar, so the direction of the step **does not change**. Use this one
- `clip_grad_value_` clamps each component separately, which **does** change the direction. On the lecture example it turned the gradient 14 degrees

Part 5: Initialization and regularization

Xavier and He

Both choose the variance of **W** so the signal keeps its size passing through a layer. They differ in the activation they assume:

	Variance	For
Xavier / Glorot	$2/(n_{\text{in}} + n_{\text{out}})$	tanh, sigmoid
He / Kaiming	$2/n_{\text{in}}$	ReLU and its relatives

Both formulas have a 2 in the numerator. What makes He larger is the **denominator**: n_{in} rather than $n_{\text{in}} + n_{\text{out}}$. With equal fan-in and fan-out, He gives exactly twice the variance.

Be able to say why He wants the larger variance: ReLU sets about half its inputs to zero, so about half the variance is lost on the way through a layer, and the larger starting variance puts it back. The formulas are on the formula sheet; the reason is not.

The regularizers, and what each one actually buys

Lever	What it does
Early stopping	Keep the best epoch's weights and stop when validation stops improving. The cheapest one, and always worth having
Weight decay	Adds $\lambda \sum_j w_j^2$ to the cost, so every step pulls weights toward zero unless the data pushes back
Dropout	Zeroes a random fraction of hidden units each training batch, so no single unit can be relied on

Details that get asked about:

- Early stopping needs **patience**, so one noisy epoch does not end the run, and it needs you to **restore** the saved weights. Save a `.clone()`; `state_dict()` hands back references to the live tensors, so an uncloned copy keeps changing underneath you
- Dropout goes **after** the activation, never after the output layer, and p between 0.1 and 0.5. `model.eval()` is what switches it off

- **Adam(weight_decay=...)** is not AdamW. In plain Adam the penalty is pushed through the same adaptive rescaling as the gradient, so parameters with small gradients get decayed harder, which is not what the formula says. AdamW applies it separately. Use AdamW

What Lab 4 found: regularization barely moved the **best** validation loss, but it clearly flattened the climb after it. It did not make the best achievable model better; it made the run **stable**, so you no longer have to catch one specific epoch. And weight decay at 10^{-3} did nothing useful on that problem at all. Each regularizer is a setting you tune, not a good you switch on.

Reading a training curve

Training cost falling while validation cost rises is the whole diagnosis: the model is learning things that are true of the examples it has seen and false of everything else.

One subtlety from Lab 4 that is worth being ready for. **Validation accuracy can still be rising while validation loss is getting worse**, on the same data at the same moment. Accuracy only asks whether the top score was the right class. Cross-entropy also asks how confident you were. As training continues the model gets more confident about everything, which nudges accuracy up a little and drives loss up a lot, because a confidently wrong answer is punished far harder than an unsure one. **The loss is the earlier warning**, which is why you stop on it.

Before you debug anything else

Overfit 100 examples first. It takes fifteen seconds. A working pipeline drives the loss to nearly zero and accuracy to 100%, because any real model can memorize 100 examples. If it cannot, the problem is not your learning rate: the labels are misaligned with the inputs, or the gradients are not reaching the weights. You have separated “my model is not learning” from “my pipeline is broken” before spending an hour.

An untrained classifier over C classes should score about $\ln C$. For ten classes that is 2.303. If your untrained loss is far from that, something is already wrong. The **accuracy** at initialization is not a useful check.

Checklist

Before Thursday, confirm you can:

- ☐ Work out the result shape of a broadcast, and say why $(N,1)$ against $(N,)$ is a bug
- ☐ Say why `.grad` accumulates, and what goes wrong without `zero_grad()`
- ☐ State what `model.eval()` does and what `torch.no_grad()` does, separately
- ☐ Say what goes on the last layer of a classifier, and what does not
- ☐ Write the five lines of the training loop in order
- ☐ Turn N , B and an epoch count into a number of optimizer steps
- ☐ Say how many optimizer steps a validation pass takes
- ☐ Give the shapes of `weight` and `bias` for an `nn.Linear(in, out)`
- ☐ Say how gradient noise scales with batch size, and what doubling B buys
- ☐ Say when `num_workers` helps and when it hurts, and why
- ☐ Recognize a data-bound training run from GPU utilization
- ☐ Define leakage and give two ways it happens
- ☐ Say why Adam’s learning rate is 100 times smaller than SGD’s
- ☐ Say which clipping method preserves the direction of the step
- ☐ Give the Xavier and He variances, and say why He has the extra 2
- ☐ Say what early stopping needs besides a stopping rule
- ☐ Say why AdamW is not `Adam(weight_decay=...)`
- ☐ Read a training script and find a missing `zero_grad()`, a missing `eval()`, and a stray softmax
- ☐ Arrive with a calculator and nothing else; no notes are permitted